

Syntaxe Matlab

VARIABLES

Scalaire

```
n = 2;           % entier
a = 2.2;         % flottant
c = 'z';         % caractère
a               % affichage de la
                % valeur de la variable a
                % dans la fenêtre de
                % commande
```

Tableaux

```
M=[1 2 3 ; 4 5 6 ; 7 8 9] ; %définition
                             % en « dure » d'une
                             % matrice 3x3

lst = 1:10 ;               % définition d'un liste
                             % d'entiers de 1 à 10

V(lst) = a ;               % définition d'un
                             % vecteur ligne de 10
                             % éléments ayant la
                             % valeur a

M(lst,lst) = a             % définition d'une
                             % matrice de 10x10
                             % éléments ayant la
                             % valeur a

M(5,4)                    %valeur de l'élément de
                             % la matrice Mat à la
                             % 5eme ligne et 4eme
                             % colonne. Les indices
                             % démarrent à 1 dans
                             % Matlab

M(:,2)                    % affichage de valeur
                             % de la 2eme colonne

V = M(4,:);               % extraction de la 4eme
                             % ligne de M et
                             % affectation à V

Chaine = 'hello world'; % définition
                             % d'une chaine de
                             % caractère (string) =
                             % tableau de caractères

Vt = transpose(V) ;        % transposition du
                             % vecteur V et
                             % affectation dans Vt

Vcc = V';                 % complexe conjugué de
                             % V et affectation à Vcc
```

Pour tableaux, utiliser aussi les fonctions Matlab et surtout les affectations par boucle for

Structures

```
VarStruct.n=1;
VarStruct.V(1)=0;
VarStruct.c='a'; %définition d'une
                 % variable structurée
                 % constituée d'un champ
                 % scalaire, d'un champ
                 % tableau et d'un champ
                 % caractère.

VarStruct.V       % affiche le contenu du
                 % champ vecteur v de la
                 % variable structurée
                 % VarStruct
```

OPERATIONS

Arithmétiques

```
a = a+2; b = a-3; d = a*b; e = a/b
% opérations + - * / sur scalaire et
% affectation dans variable scalaire
```

Vectorielles

```
V2 = V+2; %ajoute 2 à tous les éléments
           % de V et affecte à V2

M3 = M1 .* M2; % produit element par
               % element et affectation aux
               % elements de M3 (par ex. M3(5,7) =
               % M1(5,7)*M2(5,7))

M3 = M1 * M2; % produit matriciel de M1
               % par M2, affecté à M3

Attention, pour tableau et matrice, * n'est pas équivalent à .*

X = M \ V; % résolution de M X = V
```

Logique

```
a == b %renvoie vrai (1) si a est égal à
        b, faux (0) sinon
a < b ; a > b ; a <= b ; a >= b % idem
a ~= b %symbole "différent de"
(a~=b) || (a==b) %renvoie l'opération
                % logique (a différent de b) OU (a
                % égal b)
(a~=b) && (a==b) %renvoie l'opération
                % logique (a différent de b) ET (a
                % égal b)
```

STRUCTURES CONDITIONNELLES

Condition Si, Alors, Sinon

```
if (a == b)
    %action 1 terminée par un ";"
else
    %action 2.0 terminée par un ";"
    if (a < b)
        %action 2.1 terminée par
        %un ";"
    else
        %action 2.2 terminée par
        %un ";"
    end
end
```

Branchement conditionnel

```
switch (choix)
    case 1
        %action 1 terminée par un ";"
    case 2
        %action 2 terminée par un ";"
    case 4
        %action 3 terminée par un ";"
    Otherwise
        %action 4 terminée par un ";"
end

Ici, « choix » doit être dans {1,2,4} ou autre
```

BOUCLES

Boucle incrémentale

```
for i=1:N %attention au ":" - pas de ";"
           %ou de ","
           %action répétée N fois, terminée
           %par un ";"
end
```

```

Boucle tant que
while (condition vraie)
    %action terminée par un ";"
end

```

FONCTIONS DE BASE DE TRACE GRAPHIQUE

```

plot(X, Y) ; %trace le tableau Y en
fonction du tableau X
plot(X, Y, 'o') ; idem, mais avec un
symbole 'o'
xlim(xstart, xend) ;%fixe l'axe des x
entre xstart et xend
ylim(ystart, yend) ;%fixe l'axe des y
entre ystart et yend

```

```

semilogy(X,Y) ;%comme plot, mais avec un
axe y en échelle log
semilogx(X,Y) ;%comme plot, mais avec un
axe x en échelle log
Consulter l'aide de la fonction plot (help plot) pour avoir le détail
des options de tracée

```

DEFINITION D'UNE FONCTIONS MATLAB

```

% commentaire d'aide appelé par help
function [output1,output2] =
    nomfonction(input1,input2)
%corps de la fonction
end

```

Listing de fonctions Matlab

FONCTIONS MATHEMATIQUES

```

abs(x);           % valeur absolue
cos(x);           % cosinus
sin(x);           % sinus
tan(x);           % tangente
acos(x);          % arc-cosinus
asin(x);          % arc-sinus
atan(x);          % arc-tangente
exp(x);           % exponentielle
log(x);           % logarithme Népérien
log10(x);         % logarithme base 10
sqrt(x);          % racine
x^y ;            % x à la puissance y
rand(N,1) ;       % génère un vecteur de N nombres aléatoires
mod(x,y) ;        % retourne le reste de la division entière de x par y

```

FONCTIONS DE COMMANDE MATLAB

```

clc;             % efface le contenu de la fenêtre de commande
clear x ;        % efface la variable x
clear all ;      % efface toutes les variables
close all ;      % ferme toutes les fenêtres graphiques
disp('chaine') ; % affiche la chaîne de caractères

```

FONCTIONS VECTORIELLES ET MATRICIELLES

```

zeros(N);        %(!) retourne une matrice de dimension NxN remplie de 0 (!)
zeros(N1,N2);    % retourne une matrice de dimension N1xN2 remplie de 0
zeros(N1,N2,N3); % retourne un tableau de dimension N1xN2xN3 remplie de 0

ones(N1,1) % fonction identique ,mais avec des 1
Il est préférable de toujours initialiser les tableaux à l'aide de ces fonctions pour éviter l'allocation dynamique de mémoire
min(tab);        % recherche de minimum dans tab, voir help min pour les options
max(tab);        % idem avec maximum

```

FONCTIONS EVOLUEES

```

Gestion des erreurs et des warnings
error('chaine'); % génère un message d'erreur et arrête le programme

```

AUTRES FONCTIONS